pgBackRest for PostgreSQL

Step-by-Step Backup, Archiving & Restore Procedure

Item	Value used in this guide
PostgreSQL version	16 (replace with your version)
Stanza name	main
OS	Ubuntu/Debian (RHEL/Rocky commands shown where they differ)
Backup repository	/var/lib/pgbackrest (local), with remote and S3 variants
Run commands as	postgres OS user

Important: Always test the full backup and restore cycle on a non-production server before relying on it. Verify option names against the documentation for your installed pgBackRest version (`pgbackrest help`).

Contents

- 1. How pgBackRest works
- 2. Installation
- 3. Configure pgBackRest
- 4. Configure PostgreSQL for WAL archiving
- 5. Create the stanza and verify
- 6. Take backups (full / differential / incremental)
- 7. Schedule and monitor backups
- 8. Restore procedures (full, PITR, delta, standby)
- 9. Optional: dedicated repository host
- 10. Optional: S3 repository
- 11. Optional: encryption and standby backups
- 12. Troubleshooting and cheat sheet

1. How pgBackRest Works

pgBackRest is a reliable backup and restore tool for PostgreSQL. It supports parallel and compressed backups, full / differential / incremental types, continuous WAL archiving, point-in-time recovery (PITR), backup from a standby, and cloud repositories (S3, Azure, GCS).

Term	Meaning
Stanza	Configuration for one PostgreSQL cluster (data directory, port, repo settings).
Repository	Location where backups and archived WAL are stored.
Full backup	Copy of all database files. Independent of other backups.
Differential	Changes since the last full backup. Needs that full backup to restore.
Incremental	Changes since the last backup of any type. Needs the whole chain to restore.
WAL archive	Continuous copy of WAL segments, required for consistency and PITR.

2. Installation

Install on the PostgreSQL server (and on the repository host if you use a dedicated one).

Ubuntu / Debian

```
sudo apt update
sudo apt install -y pgbackrest
pgbackrest version
```

RHEL / Rocky / AlmaLinux (PGDG repository enabled)

```
sudo dnf install -y pgbackrest
pgbackrest version
```

Create directories and set ownership

```
sudo mkdir -p /var/lib/pgbackrest /var/log/pgbackrest /etc/pgbackrest
sudo chown -R postgres:postgres /var/lib/pgbackrest /var/log/pgbackrest
sudo chmod 750 /var/lib/pgbackrest
```

Note: Debian packages may create some of these automatically. The default config file is /etc/pgbackrest.conf on Debian and /etc/pgbackrest/pgbackrest.conf on newer layouts; use whichever `pgbackrest version` / package docs indicate, and keep only one.

3. Configure pgBackRest

Edit `/etc/pgbackrest.conf`. Find your data directory with `sudo -u postgres psql -c "SHOW data_directory;"`.

```
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2
repo1-retention-diff=6
process-max=2
start-fast=y
compress-type=zst
log-level-console=info
log-level-file=detail
log-path=/var/log/pgbackrest

[main]
pg1-path=/var/lib/postgresql/16/main
pg1-port=5432
```

Option	Purpose
repo1-path	Where backups and WAL archive are stored.
repo1-retention-full	Number of full backups to keep (older ones, and their WAL, are expired).
repo1-retention-diff	Number of differential backups to keep.
process-max	Parallel processes for backup/restore. Do not exceed available CPU.
start-fast	Forces an immediate checkpoint at backup start instead of waiting.
pg1-path	PostgreSQL data directory (RHEL: /var/lib/pgsql/16/data).

```
sudo chown postgres:postgres /etc/pgbackrest.conf
sudo chmod 640 /etc/pgbackrest.conf
```

4. Configure PostgreSQL for WAL Archiving

Edit postgresql.conf (Debian: /etc/postgresql/16/main/postgresql.conf).

```
archive_mode = on
archive_command = 'pgbackrest --stanza=main archive-push %p'
wal_level = replica
max_wal_senders = 3
archive_timeout = 60        # optional: forces a WAL switch at most every 60s
```

Changing `archive_mode` or `wal_level` needs a restart:

```
sudo systemctl restart postgresql          # Debian/Ubuntu
sudo systemctl restart postgresql-16       # RHEL family
```

Warning: If `archive_command` fails, WAL files pile up in pg_wal and can fill the disk. Monitor archiving (see Section 7).

5. Create the Stanza and Verify

```
sudo -u postgres pgbackrest --stanza=main stanza-create
sudo -u postgres pgbackrest --stanza=main check
```

`stanza-create` initialises the repository for this cluster. `check` forces a WAL switch and confirms the archive pipeline works. Both must finish without errors before you take a backup.

Note: If you plan to use repository encryption, configure it **before** running stanza-create (Section 11).

6. Take Backups

Step 1 - First full backup

```
sudo -u postgres pgbackrest --stanza=main --type=full backup
```

Step 2 - Differential and incremental backups

```
sudo -u postgres pgbackrest --stanza=main --type=diff backup
sudo -u postgres pgbackrest --stanza=main --type=incr backup
```

If no type is given, pgBackRest takes an incremental backup, and automatically makes a full one when none exists.

Step 3 - Review backup information

```
sudo -u postgres pgbackrest info
sudo -u postgres pgbackrest --stanza=main --output=json info
```

Step 4 - Optional: verify repository integrity

```
sudo -u postgres pgbackrest --stanza=main verify
```

7. Schedule and Monitor Backups

Create `/etc/cron.d/pgbackrest` (weekly full, daily incremental):

```
# m h dom mon dow user command
30 2 * * 0   postgres  pgbackrest --stanza=main --type=full backup
30 2 * * 1-6 postgres  pgbackrest --stanza=main --type=incr backup
```

Monitoring checklist:

- Check `pgbackrest info` output daily or export it to your monitoring system.
- Alert on failed cron jobs and on errors in /var/log/pgbackrest/.
- Watch `pg_stat_archiver` for failures:

```
SELECT archived_count, last_archived_wal, failed_count, last_failed_wal
FROM pg_stat_archiver;
```

- Run a test restore regularly. An untested backup is not a backup.

8. Restore Procedures

Warning: Restore overwrites data. Confirm you are on the right server, and take a copy of anything you may still need first.

8.1 Full restore to the original location (empty or cleaned data directory)

```
sudo systemctl stop postgresql
sudo -u postgres find /var/lib/postgresql/16/main -mindepth 1 -delete
sudo -u postgres pgbackrest --stanza=main restore
sudo systemctl start postgresql
```

8.2 Delta restore (faster; only changed files are replaced)

```
sudo systemctl stop postgresql
sudo -u postgres pgbackrest --stanza=main --delta restore
sudo systemctl start postgresql
```

8.3 Point-in-time recovery (PITR)

Use this to recover to a moment just before a bad change (accidental DROP, bad deployment, etc.).

```
sudo systemctl stop postgresql
sudo -u postgres pgbackrest --stanza=main --delta \
     --type=time --target="2026-10-29 10:15:00+05:30" \
     --target-action=promote restore
sudo systemctl start postgresql
```

- pgBackRest picks a backup that finished before the target time and writes the recovery settings automatically.
- Watch the PostgreSQL log until recovery completes and the server accepts connections.
- Other targets: `--type=lsn`, `--type=xid`, `--type=name` (restore point), `--type=immediate`.

8.4 Restore a specific backup set

```
sudo -u postgres pgbackrest info                       # find the backup label
sudo -u postgres pgbackrest --stanza=main --delta \
     --set=20261025-020001F restore
```

8.5 Restore to a different server or directory

```
# On the new server: install same PostgreSQL major version + pgBackRest,
# copy pgbackrest.conf, and point pg1-path at the new data directory
sudo -u postgres pgbackrest --stanza=main --pg1-path=/var/lib/postgresql/16/main restore
```

8.6 Restore as a standby (replica)

```
sudo -u postgres pgbackrest --stanza=main --type=standby restore
# Then add primary_conninfo in postgresql.auto.conf and start PostgreSQL
```

8.7 Post-restore checks

- Confirm PostgreSQL started and accepts connections.
- Run application sanity queries and compare row counts if needed.
- Take a new full backup after a PITR or promotion (it starts a new timeline).
- Re-run `pgbackrest --stanza=main check`.

9. Optional: Dedicated Repository Host

Storing backups on a separate server protects against loss of the database host. pgBackRest uses SSH (or TLS in recent versions) between the hosts.

Step 1 - Install pgBackRest on both hosts and create a user on the repo host

```
# repo host
sudo useradd -m -s /bin/bash pgbackrest
sudo mkdir -p /var/lib/pgbackrest && sudo chown pgbackrest: /var/lib/pgbackrest
```

Step 2 - Passwordless SSH both ways

```
# On DB host as postgres:
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
ssh-copy-id pgbackrest@repo-host
# On repo host as pgbackrest:
ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519
ssh-copy-id postgres@db-host
```

Step 3 - Config files

Repository host /etc/pgbackrest.conf:

```
[global]
repo1-path=/var/lib/pgbackrest
repo1-retention-full=2

[main]
pg1-host=db-host
pg1-host-user=postgres
pg1-path=/var/lib/postgresql/16/main
```

Database host /etc/pgbackrest.conf:

```
[global]
repo1-host=repo-host
repo1-host-user=pgbackrest

[main]
pg1-path=/var/lib/postgresql/16/main
```

Step 4 - Create stanza and back up from the repo host

```
sudo -u pgbackrest pgbackrest --stanza=main stanza-create
sudo -u pgbackrest pgbackrest --stanza=main check
sudo -u pgbackrest pgbackrest --stanza=main --type=full backup
```

Keep `archive_command` on the DB host as in Section 4; it pushes WAL to the repo host automatically. Restores are run on the DB host as the postgres user.

10. Optional: S3 Repository

```
[global]
repo1-type=s3
repo1-path=/pgbackrest
repo1-s3-bucket=my-pg-backups
repo1-s3-endpoint=s3.amazonaws.com
repo1-s3-region=ap-south-1
repo1-s3-key=AKIA................
repo1-s3-key-secret=********************
repo1-retention-full=4
```

- On AWS, prefer an IAM role: set `repo1-s3-key-type=auto` and omit the key and secret.
- For S3-compatible storage (MinIO, etc.) set the endpoint accordingly; add `repo1-storage-verify-tls` and `repo1-s3-uri-style=path` if required.
- Run stanza-create, check and backup exactly as before.

11. Optional: Encryption and Standby Backups

Repository encryption

Set before `stanza-create`. It cannot be changed on an existing repository.

```
# generate a strong passphrase
openssl rand -base64 48

[global]
repo1-cipher-type=aes-256-cbc
repo1-cipher-pass=<paste-passphrase-here>
```

Critical: Store the passphrase safely outside the server. Without it the backups cannot be restored.

Backup from a standby (reduces primary load)

```
[global]
backup-standby=y

[main]
pg1-host=primary-host
pg1-path=/var/lib/postgresql/16/main
pg2-host=standby-host
pg2-path=/var/lib/postgresql/16/main
```

pgBackRest still contacts the primary for start/stop backup and WAL, but copies data files from the standby.

12. Troubleshooting and Cheat Sheet

Symptom	Likely cause / fix
check fails: archive timeout	archive_command wrong, archive_mode off, or PostgreSQL not restarted. Test with `pgbackrest --stanza=main archive-push` and read the PostgreSQL log.
Permission denied	Repo, log directory or config not owned by postgres (or pgbackrest user on repo host).
stanza-create: path mismatch	pg1-path does not match the running cluster; confirm with SHOW data_directory.

Symptom	Likely cause / fix
pg_wal filling up	Archiving failing. Fix archive_command; check repo disk space; review pg_stat_archiver.
Restore: data directory not empty	Use --delta, or clear the directory first.
Recovery does not reach target	Missing WAL in the archive, or target is before the backup ended. Choose an earlier backup set with --set.
Version mismatch error	PostgreSQL was upgraded; run `stanza-upgrade`.

Command cheat sheet

Task	Command
Create stanza	pgbackrest --stanza=main stanza-create
Check config/archive	pgbackrest --stanza=main check
Full backup	pgbackrest --stanza=main --type=full backup
Differential backup	pgbackrest --stanza=main --type=diff backup
Incremental backup	pgbackrest --stanza=main --type=incr backup
List backups	pgbackrest info
Verify repository	pgbackrest --stanza=main verify
Restore latest	pgbackrest --stanza=main --delta restore
Restore to time	pgbackrest --stanza=main --delta --type=time --target="..." restore
Expire per retention	pgbackrest --stanza=main expire
Upgrade stanza	pgbackrest --stanza=main stanza-upgrade
Delete stanza	pgbackrest --stanza=main --force stop && pgbackrest --stanza=main stanza-delete

Official documentation: https://pgbackrest.org